Equipment Loan Manager

Design and Implementation Report

1. Overview

Equipment Loan Manager is a Windows desktop app built in C# with WPF. It simulates how a campus lends out equipment such as laptops, 360 cameras, VR headsets, microphones and tripods. Nothing is saved to a database. All data lives in memory for one session, so a tester can try the borrowing rules freely and start again from a clean state by reopening the app.

A tester uses the app like this:

1. **Set up stock.** On startup a setup window asks how many items of each category exist. Each category has a maximum (for example 8 laptops, 12 tripods). Only whole numbers from 0 up to that maximum are accepted, and bad entries are highlighted. The app then creates every item with its own ID, such as `LAP-01`.
2. **Add borrowers.** On the Borrowers tab the tester types a name and, if they want, an ID. If the ID is left blank the app makes one, such as `BR-001`.
3. **Borrow and return.** On the Loans tab the tester picks a borrower and a category and clicks Borrow. The app finds a free item and creates a loan. Selecting an active loan and clicking Return gives the item back. Returned loans stay in a history list.
4. **Use waiting lists.** When every item in a category is out, the borrower can join that category's queue. When an item comes back, the tester issues it to the person at the front of the queue.
5. **Check the inventory.** The Inventory tab shows how many items are configured, available and borrowed, and who holds each one.

After every action a banner at the top shows the result: green when it worked and red with a clear reason when it was refused (for example, "Amara already has 2 active loans"). The main rules are: a borrower can hold at most two items and only one per category, an item can never be lent twice at once, a loan cannot be returned twice, and nobody can skip a waiting list.

The solution has three projects. **Core** holds all the rules and has no UI code. **App** is the WPF front end, built with the Model-View-ViewModel (MVVM) pattern. **Tests** holds xUnit tests that run against Core. The UI only talks to one class, `LoanManager`, which acts as a single entry point (a facade) to the rest of Core.

2. UML Class Diagram

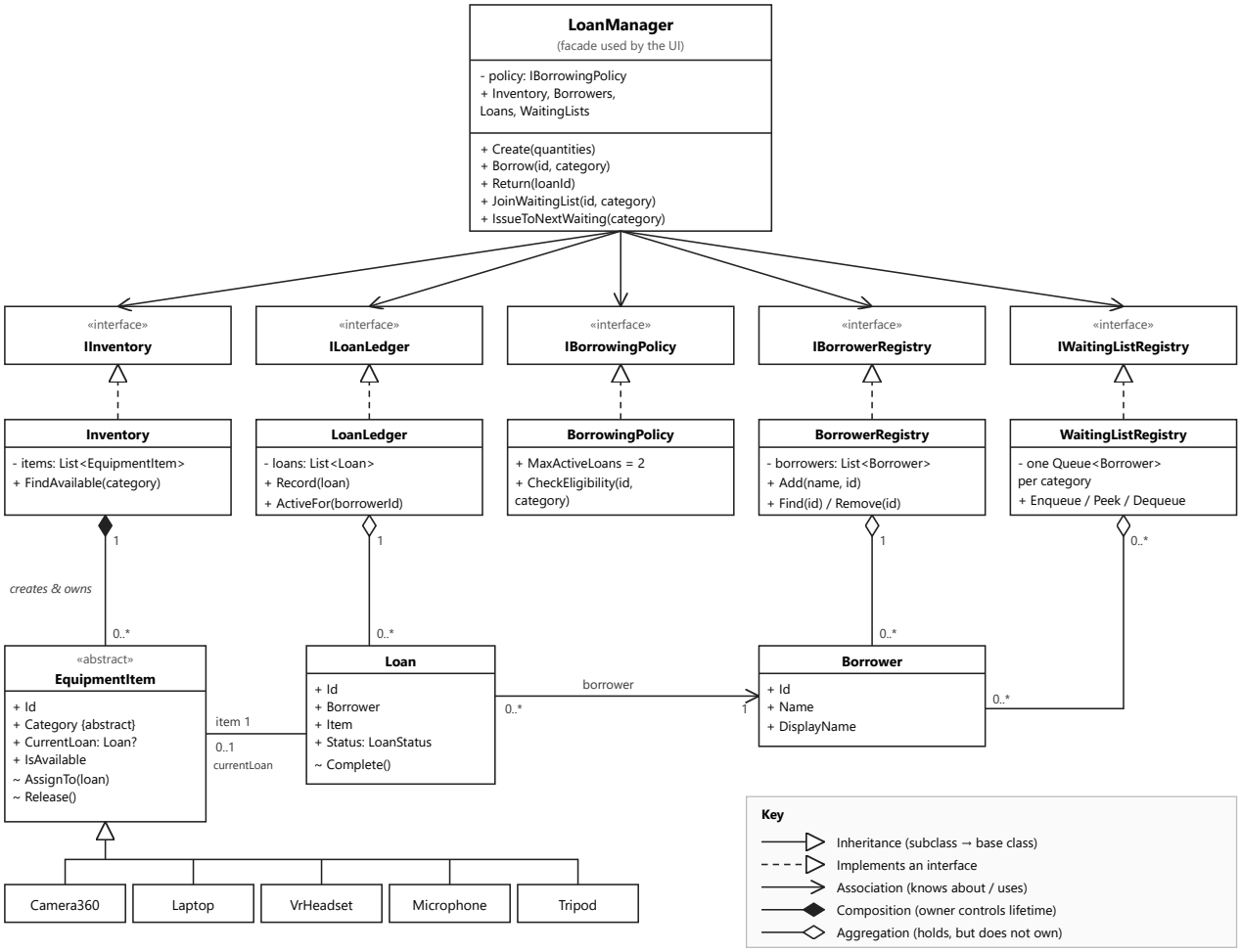


Figure 1. Main classes and interfaces in the Core project. Only key members are shown.

The diagram shows the Core project only, since that is where the design decisions are. `LoanManager` depends on five interfaces rather than on concrete classes, and each interface has one implementation. `BorrowingPolicy` also reads from `IBorrowerRegistry` and `ILoanLedger` to check the rules; that link is left out to keep the picture readable. On the App side, each tab has a view model (for example `LoansViewModel`) that inherits from `SectionViewModel` and calls `LoanManager`.

3. Relationship Explanations

Inheritance: EquipmentItem and its subclasses

`EquipmentItem` is an abstract class. `Laptop`, `Camera360`, `VrHeadset`, `Microphone` and `Tripod` inherit from it. Every item, whatever its type, needs the same things: an ID, a current loan, an "is it available" check, and safe ways to lend and release it. Putting this in the base class means the lending logic is written once. Each subclass only says which category it belongs to.

```
public abstract class EquipmentItem
{
    public abstract EquipmentCategory Category { get; }
    public bool IsAvailable => CurrentLoan is null;
    internal void AssignTo(Loan loan) { ... }
}

public sealed class Laptop(string id) : EquipmentItem(id)
{
    public override EquipmentCategory Category => EquipmentCategory.Laptop;
}
```

This is a true "is-a" relationship: a laptop is a piece of equipment. The rest of the code, such as `Inventory` and `Loan`, works with `EquipmentItem` and never needs to know the exact type. Adding a new kind of equipment means adding one small subclass and one line in the factory and catalog.

Association: Loan and Borrower

A `Loan` holds a reference to one `Borrower` and one `EquipmentItem`. A borrower can appear in many loans over time. This is an association, not ownership. The loan just records who borrowed what. It did not create the borrower and does not control when the borrower exists.

This matters when a borrower is removed. `BorrowerRegistry` drops them from its list, but their completed loans still point to the same `Borrower` object, so the loan history stays readable. Because the reference is kept, the borrower object stays alive as long as some loan still needs it. A test (`RemoveBorrower_PreservesCompletedLoanHistory`) checks this.

Composition: Inventory and EquipmentItem

`Inventory` creates every `EquipmentItem` itself, in its constructor, using the quantities from the setup screen. No other class can create or delete items. The list is private and only exposed as read-only. The items are made when the inventory is made and are never used outside it. So the inventory owns its items, and they share its lifetime. That is composition.

The whole object graph has a clear lifetime. `LoanManager.Create` builds the inventory, registries and ledger when the tester clicks Start on the setup screen. They all live until the main window closes, and then everything is released together. Nothing is shared between sessions.

By contrast, `WaitingListRegistry` and `Borrower` is an aggregation. The queues hold references to borrowers, but they did not create them. Removing someone from a queue does not delete the borrower; it only cancels their request.

4. Testing

The Tests project has **50 automated xUnit tests**, and all of them pass. They test Core directly through `LoanManager`, without opening any windows, so they run in well under a second. A small helper, `TestSession`, sets up a session in one line, for example `TestSession.Create(laptops: 1)`.

Test file	What it covers

ConfigurationTests	Category limits; quantity input rules (blank, negative, decimals, text, too large)
InventoryTests	Correct number and type of items, unique IDs, summary counts
BorrowerTests	Generated IDs, empty names, duplicate IDs, removing borrowers safely
LoanTests	Borrowing, the two-item and one-per-category limits, returns and double returns
WaitingListTests	Joining, cancelling, first-in-first-out order, no queue skipping, rechecking eligibility

TDD cycle 1: limit of two active loans

Red. We wrote `Borrow_RejectsThirdActiveLoan` first. It gives one borrower a laptop and a headset, then asks for a tripod and expects a refusal. It failed, because `Borrow` handed out any free item.

```
manager.Borrow(borrower.Id, EquipmentCategory.Laptop);
manager.Borrow(borrower.Id, EquipmentCategory.VrHeadset);
var result = manager.Borrow(borrower.Id, EquipmentCategory.Tripod);
Assert.False(result.Succeeded);
```

Green. We added a check that counts the borrower's active loans and refuses at two. The test passed.

Refactor. Joining a waiting list needed the same rule, so we moved it out of `LoanManager` into `BorrowingPolicy` behind `IBorrowingPolicy`. All the tests still passed. The next test, `Borrow_RejectsSecondItemFromSameCategory`, then only needed one extra line in the policy.

TDD cycle 2: nobody can skip the queue

Red. We wrote `Borrow_CannotBypassExistingQueue`. One person waits for a laptop, the laptop is returned, and then a different person tries to borrow it directly. The test expects a refusal. It failed, because `Borrow` saw a free laptop and lent it to the latecomer.

Green. We made `Borrow` refuse when anyone is already waiting for that category, with a message telling the tester to join the list instead.

```
var waiting = WaitingLists.GetQueue(category).Count;
if (waiting > 0)
    return OperationResult<Loan>.Failure(...);
```

Refactor. This showed that the person at the front of the queue could have become ineligible while waiting. We added `IssueToNext_RechecksEligibilityBeforeIssuing` and made `IssueToNextWaiting` reuse the same policy check before lending.

5. Reflection

Team contributions

Member	Contribution

[Name 1]	[e.g. Core models, Inventory and configuration rules]
[Name 2]	[e.g. LoanManager, BorrowingPolicy and waiting lists]
[Name 3]	[e.g. WPF views, view models and styling]
[Name 4]	[e.g. Unit tests, TDD cycles and this report]

Design challenge

The hardest part was keeping the screens in step with each other. One action, such as a return, changes the inventory counts, the loan lists, the waiting lists and the header totals all at once. Our first idea was for each tab to update itself, but that got messy fast and tabs often showed old data. We solved it by sending every result through a single method, `MainViewModel.Show`. It shows the message, then tells every tab to refresh from `LoanManager`. This gives one simple path, and the screen always matches the real state.

Potential improvement

The biggest gap is that nothing is saved. When the app closes, every borrower and loan is lost. Because Core already works through interfaces such as `ILoanLedger` and `IBorrowerRegistry`, we could add versions that save to a file or a small SQLite database without changing `LoanManager` or the UI. Due dates and overdue warnings would be a natural next step after that.